

Improved Priority Exchange Server: Technical Briefing

Milestone 1 — Special Emphasis: Real-Time Systems, HSHL

Sumon Boiddo

BEng in Electronic Engineering

Hochschule Hamm-Lippstadt

Matriculation Number: 2220054

Group B, Team 6

Abstract—This document is the Milestone 1 technical briefing for the seminar topic *Improved Priority Exchange Server (IPE)*. It explains the problem that motivates IPE, its core idea of reusing precomputed idle times, the formal definition with its replenishment mechanism, the scheduling rules inherited from the Dynamic Priority Exchange server, the schedulability condition, the underlying assumptions, the run-time and memory overhead, the relevance of IPE for embedded systems, and the open questions for further study. The primary reference is Buttazzo, *Hard Real-Time Computing Systems* [1], Chapter 6.

I. PROBLEM STATEMENT AND MOTIVATION

- A real-time system runs two kinds of tasks together. *Periodic* tasks arrive at fixed, regular intervals that are known in advance — for example a temperature sensor reading data every second, or a task released every 10 ms — and they have *hard* deadlines that must never be missed [1].
- *Aperiodic* tasks arrive at random, unpredictable times — for example an emergency such as an alarm or a sudden external request. They have soft deadlines and should be served as fast as possible.
- There are two simple ways to handle them, but both are bad. If the aperiodic task runs only in the *background*, it waits until all periodic tasks finish, so when the system is busy its response time becomes very poor. If instead it runs at the *highest priority*, it keeps delaying the periodic tasks, which can make them miss a hard deadline — and in a hard real-time system that means **system failure**.
- A better foundation is the *Earliest Deadline First* (EDF) scheduler. With many tasks, Rate Monotonic (RM) can safely use only about 69% of the CPU, but EDF can use up to 100%. Because more CPU is available under EDF, a larger aperiodic server can be used, so aperiodic requests receive better service [1].
- Two EDF servers already exist. The *Dynamic Priority Exchange* (DPE) server does not waste its free time: when no aperiodic request is waiting, it lends its time to a periodic task and saves an equal amount to reuse later [3]. The *Earliest Deadline Late* (EDL) server is *optimal* — it gives the fastest possible aperiodic response — but it

must recompute the schedule at run time, which makes it too expensive for practical use [2].

- **Core problem:** how can a server reach the near-optimal responsiveness of EDL *without* paying EDL's heavy run-time cost? The Improved Priority Exchange (IPE) server answers this by computing the EDL idle times *off-line*, *before running*, and reusing them inside a DPE-style server — keeping EDL's good result while removing the expensive part.

II. CORE TECHNICAL CONTRIBUTION

- IPE is a dynamic-priority (EDF) aperiodic server. Its main contribution is the way it *combines two existing servers*: it uses the working method of the DPE server (the priority-exchange mechanism) together with the idle times of the optimal EDL server [3].
- The key idea is to calculate the EDL idle times *off-line*, *before the system runs*, and store them. This is comparable to preparing a complete schedule in advance: once the schedule is ready, no further decisions or calculations are needed while the work is carried out. In the same way, IPE performs its timing calculation beforehand, so that at run time it can serve an aperiodic (emergency) task very quickly, without expensive recomputation.
- Because the heavy work is done off-line, IPE avoids the high run-time cost of the EDL server while still serving aperiodic requests fast.
- IPE keeps the priority-exchange mechanism of DPE: when no aperiodic request is waiting, the server lends its free time to a periodic task and saves an equal amount of capacity to reuse later, so no CPU time is wasted.
- As a result, IPE behaves almost the same as the optimal EDL server — it is *nearly optimal* — but at a much lower, practical run-time cost [1].

III. FORMAL DEFINITION

IPE is built on the idea of *idle time* — the moments when the CPU is free. This is described by the availability function

$\omega(t)$:

$$\omega(t) = \begin{cases} 1 & \text{if the CPU is idle (free) at time } t \\ 0 & \text{if the CPU is busy at time } t. \end{cases} \quad (1)$$

The total idle time in an interval $[t_1, t_2]$ is the sum of all the free moments in that interval:

$$\Omega(t_1, t_2) = \int_{t_1}^{t_2} \omega(t) dt. \quad (2)$$

IPE uses the idle times of the EDL schedule, which has a key property: among all scheduling methods, EDL leaves the *maximum* free time available as early as possible [5]. Because the free time comes early, an arriving aperiodic request is served sooner, its waiting time is reduced, and the periodic deadlines stay safe.

The pattern of EDL free times repeats every hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$ (the least common multiple of all task periods). The free times are therefore calculated only once, off-line, for a single hyperperiod, and stored in two arrays:

- $E = (e_0, e_1, \dots, e_p)$ — the times at which the free periods occur (“when”);
- $D = (\Delta_0, \Delta_1, \dots, \Delta_p)$ — the lengths of those free periods (“how long”).

The IPE server is then defined by the following rules [3]:

- the server has an aperiodic capacity, initially set to 0;
- at each instant $t = e_i + kH$ (with $k = 0, 1, 2, \dots$), the capacity is replenished by Δ_i units — exactly the stored free time;
- the server always runs at the highest priority in the system;
- all other rules (serving aperiodic requests, executing periodic tasks, and the exchange and saving of capacity) are the same as in the DPE server.

Schedulability condition. The periodic tasks together with the IPE server are schedulable (no hard deadline is ever missed) if and only if

$$U_p \leq 1, \quad (3)$$

which is necessary because the CPU cannot do more than 100% of work. The server automatically allocates the remaining bandwidth $1 - U_p$ to the aperiodic requests [3].

IV. ALGORITHM AND EXCHANGE/REPLENISHMENT RULES

This section describes, step by step, how the IPE server works.

- **Replenishment.** The server starts with capacity 0. At the precomputed times $t = e_i + kH$, it is refilled (replenished) by Δ_i units — exactly the EDL free time stored in the arrays E and D .
- **Exchange (no aperiodic request waiting).** When the server has capacity but no aperiodic task is waiting, the time is not wasted: a periodic task runs now, and an equal amount of capacity is saved (exchanged), so the server can use it later when an aperiodic request arrives. In this way no CPU time is wasted.

- **Consumption (aperiodic request arrives).** When an aperiodic request arrives and the server has capacity, it is served immediately, because the server always runs at the highest priority. As the request runs, the capacity decreases — each unit of execution uses one unit of capacity. When the capacity reaches 0, the server is exhausted and can no longer serve aperiodic requests; the periodic tasks run again, and the server waits for the next replenishment.
- **Spare-time reclaiming.** When a periodic task finishes earlier than its planned (worst-case) time — for example, it was given 10 units but finished in 5 — the leftover time is not wasted. It is added to the server’s capacity so an aperiodic request can use it. This is safe and breaks no periodic deadline, because that time was already reserved for the periodic task [1].

V. ASSUMPTIONS AND SCOPE LIMITATIONS

- The complete periodic task set must be known in advance (off-line), before the system runs — each periodic task with its period and its computation time. This is required so that IPE can precompute the free-time arrays E and D off-line.
- The system uses a single processor (one CPU). The basic IPE model is defined for one CPU, not for multiprocessors.
- All tasks are fully preemptive — a running task can be paused and continued later. This matters because an aperiodic request must be able to interrupt a running periodic task and run immediately.
- For the aperiodic tasks, the arrival time is unknown (they come suddenly and randomly), but their computation time (worst-case execution time, WCET) is known.
- Each periodic task has a period T_i , a computation time C_i , and a relative deadline equal to its period ($D_i = T_i$). All periodic tasks are assumed to be activated together at time $t = 0$.
- Some of these assumptions can be relaxed with a modified analysis — for example arbitrary task phasing, deadlines different from periods, and shared resources (handled by the Stack Resource Policy). However, the single-processor and off-line-known task-set assumptions are fundamental to the basic IPE method [1].

VI. RUNTIME AND OVERHEAD

- The heavy calculation — the E and D arrays — is done off-line, before the system runs. So at run time IPE only uses the stored result and does not recalculate anything. This makes its run-time cost low and fast.
- Compared to the EDL server, which must recompute the schedule at run time (high cost, slow), IPE moves all that heavy work off-line. As a result, its run-time overhead is low — comparable to the DPE server [1].
- The main cost of IPE is *memory*. IPE must store the E and D arrays, and their size depends on the hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$.

- If the periods fit nicely together (harmonic), H is small, so the tables are small and need little memory. But if the periods do not fit nicely (non-harmonic), H becomes very large, the stored tables become very large, and a lot of memory is needed. This is the main practical limitation of IPE [1].

VII. RELEVANCE FOR EMBEDDED SYSTEMS

- IPE is a good choice for embedded systems where the periodic task set is fixed and known in advance (static). Because everything is known beforehand, IPE can precompute the E and D arrays off-line and then serve aperiodic requests very fast at run time.
- It works best when the periods are nicely matched (harmonic), so the hyperperiod stays small and the stored tables need little memory; when the platform supports an EDF scheduler; and when the system needs fast response to aperiodic events while still guaranteeing all hard periodic deadlines.
- IPE is a poor choice when the tasks are not known in advance — that is, in a dynamic system where the task set changes while running — because then the E and D arrays cannot be precomputed.
- It is also a poor choice when the periods do not fit nicely (non-harmonic), because the hyperperiod becomes huge and the stored tables need too much memory; and when the platform supports only fixed priority (RM) with no EDF scheduler.

VIII. OPEN QUESTIONS

- **Memory limitation.** How large can the memory cost of the E and D arrays actually become for realistic non-harmonic task sets, and can it be reduced or bounded?
- **Comparison with EDL.** How close is IPE to the optimal EDL server in practice, and in which load conditions does EDL still perform slightly better?
- **Comparison with other servers.** How does IPE compare with simpler servers (such as the Total Bandwidth Server) in terms of responsiveness versus implementation simplicity?
- **Relaxing the assumptions.** What happens if the base assumptions are relaxed — for example a changing (dynamic) task set, deadlines different from periods, or shared resources?

REFERENCES

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011, ch. 6 (Dynamic Priority Servers).
- [2] M. Spuri and G. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1994.
- [3] M. Spuri and G. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, no. 2, March 1996.
- [4] J. Lehoczky, L. Sha, and J. Strosnider, "Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1987.
- [5] H. Chetto and M. Chetto, "Some Results of the Earliest Deadline Scheduling Algorithm," *IEEE Trans. Software Engineering*, vol. 15, no. 10, 1989.
- [6] S. Boiddo, *AI Usage Protocol, Milestone 1: Improved Priority Exchange Server*, HSHL, 2026. Protocol version 0.2; JSON and BibTeX files submitted with this milestone.